

Troubleshooting & Diagnostics

Solving Migration Issues Fast

95% of migration issues fall into 10 common categories. This guide provides immediate diagnosis and fix for every known failure mode — from BSOD on Windows VMs to initramfs panics on Linux, from UEFI boot failures to network-after-migration issues. Includes h2kvmctl doctor output interpretation and log analysis.

Boot Failures | BSOD Fix | initramfs Panic | UEFI/BIOS | doctor.sh | Log Analysis | 10 Common Issues

Top 10 Migration Issues

Quick reference: symptom → cause → fix for the most common problems.

#	Symptom	OS	Root Cause	Fix	h2kvmctl Flag
1	BSOD 0x7B INACCESSIBLE_BOOT_DEVICE	Windows	Missing VirtIO SCSI/block driver	Inject VirtIO drivers before boot	<code>--virtio-drivers /path/virtio-win.iso</code>
2	Kernel panic: no init found	Linux	initramfs missing virtio modules	Rebuild initramfs with virtio drivers	<code>--fix-initramfs</code>
3	VM won't boot (black screen)	Both	UEFI VM booted in BIOS mode (or vice versa)	Match firmware type to source VM	<code>--firmware uefi</code> OR <code>--firmware bios</code>
4	Filesystem mount failed	Linux	fstab references old device names (sda → vda)	Convert fstab to UUID-based entries	<code>--fix-fstab</code>
5	No network after boot	Both	NIC changed from e1000/vmxnet3 to virtio	Regenerate network config for new NIC	<code>--fix-network</code>
6	Slow disk I/O	Both	Using IDE emulation instead of VirtIO	Change disk bus to virtio in domain XML	Auto (h2kvmctl uses virtio by default)
7	GRUB rescue prompt	Linux	GRUB config references old disk layout	Regenerate GRUB config	<code>--fix-grub</code>
8	Windows activation lost	Windows	Hardware fingerprint changed	Re-activate or use KMS server	N/A (Windows issue, not h2kvmctl)
9	Conversion error: qemu-img failed	N/A	Corrupt source VMDK or insufficient space	Check source integrity, free disk space	<code>h2kvmctl doctor</code> checks space
10	VM starts but no console output	Both	Serial console not configured, video driver mismatch	Add virtio-vga or QXL video device	Auto (h2kvmctl adds appropriate video)

Windows Boot Failures (BSOD)

The #1 migration issue: Windows BSOD 0x7B — and how h2kvmctl prevents it.

BSOD Prevention Workflow

Why BSOD Happens

- VMware uses LSI Logic SAS or PVSCSI virtual disk controller
- KVM uses VirtIO SCSI (vioscsi) or VirtIO Block (viorstor)
- Windows doesn't have VirtIO drivers in-box
- At boot, Windows can't find the boot disk → BSOD 0x7B
- This is the most common migration failure (affects 100% of Windows VMs without fix)
- Solution: inject VirtIO drivers BEFORE first boot

h2kvmctl Auto-Fix

```
# Automatic VirtIO injection
h2kvmctl convert \
  --source win-server.vmdk \
  --format qcow2 \
  --virtio-drivers /usr/share/virtio-win/virtio-win.iso \
  --emit-domain-xml --deploy-libvirt

# What h2kvmctl does internally:
# 1. Mounts QCOW2 with qemu-nbd
# 2. Loads Windows registry hive (SYSTEM)
# 3. Adds viostor/vioscsi driver entries
# 4. Copies .sys/.inf files to drivers dir
# 5. Marks drivers as boot-critical
# 6. Unmounts cleanly
# 7. Result: Windows boots with VirtIO
```

Manual Fix (if auto-fix fails)

```
# Mount VM disk
modprobe nbd max_part=8
qemu-nbd -c /dev/nbd0 win-server.qcow2

# Mount Windows partition
mount /dev/nbd0p2 /mnt/win

# Copy VirtIO drivers
mkdir -p /mnt/win/drivers/virtio
cp /mnt/virtio-win/viostor/2k22/amd64/* \
  /mnt/win/drivers/virtio/
cp /mnt/virtio-win/vioscsi/2k22/amd64/* \
  /mnt/win/drivers/virtio/

# Load registry and add driver
# (h2kvmctl handles this automatically)
```

VirtIO Driver Versions

Windows Version	Driver Path
Server 2025	viostor/2k25/amd64
Server 2022	viostor/2k22/amd64
Server 2019	viostor/2k19/amd64
Server 2016	viostor/2k16/amd64
Windows 11	viostor/w11/amd64
Windows 10	viostor/w10/amd64

Download: `dnf install virtio-win` (Fedora/RHEL)

```
hivex /mnt/win/Windows/System32/config/SYSTEM \  
--add-driver viostor
```

```
# Cleanup  
umount /mnt/win  
qemu-nbd -d /dev/nbd0
```

Linux Boot Failures

Kernel panics, mount failures, and GRUB issues — all fixable offline.

initramfs Kernel Panic

```
# Symptom: "Kernel panic - not syncing: VFS: Unable to mount root fs"
# Or: "ALERT! /dev/sda1 does not exist. Dropping to a shell!"

# Cause: initramfs was built with VMware SCSI drivers (mptspi, vmw_pvscsi)
# but doesn't contain VirtIO drivers (virtio_blk, virtio_scsi, virtio_pci)

# h2kvmctl fix (automatic):
h2kvmctl convert --source linux-vm.vmdk --fix-initramfs

# What it does internally:
# 1. Mounts guest filesystem with qemu-nbd
# 2. Chroots into the guest OS
# 3. Detects distro (RHEL, Ubuntu, SUSE, etc.)
# 4. Runs appropriate initramfs rebuild:
#   RHEL/Rocky: dracut --force --add-drivers "virtio_blk virtio_scsi virtio_net virtio_pci"
#   Ubuntu:    update-initramfs -u -k all
#   SUSE:      mkinitrd -m "virtio_blk virtio_scsi"
# 5. Verifies virtio modules are in new initramfs:
#   lsinitrd /boot/initramfs-*.img | grep virtio
```

fstab Device Name Issues

Problem: Device Names Change

```
# VMware uses SCSI emulation:
# /dev/sda1 → /boot
# /dev/sda2 → swap
# /dev/sda3 → /

# KVM with VirtIO uses:
# /dev/vda1 → /boot
# /dev/vda2 → swap
# /dev/vda3 → /

# If fstab says /dev/sda1 → mount fails!
```

Fix: Convert to UUID

```
# h2kvmctl --fix-fstab converts to UUIDs:

# Before (broken):
/dev/sda1 /boot ext4 defaults 0 1
/dev/sda2 swap swap defaults 0 0
/dev/sda3 / xfs defaults 0 1

# After (works everywhere):
UUID=a1b2c3d4 /boot ext4 defaults 0 1
UUID=e5f6a7b8 swap swap defaults 0 0
UUID=c9d0e1f2 / xfs defaults 0 1
```

GRUB Recovery

```
# Symptom: "error: no such partition" or GRUB rescue shell
# Cause: GRUB references (hd0,msdos1) which changed with disk controller
```

```
# h2kvmctl --fix-grub rebuilds GRUB config:
# BIOS: grub2-install /dev/vda && grub2-mkconfig -o /boot/grub2/grub.cfg
# UEFI: grub2-mkconfig -o /boot/efi/EFI/*/grub.cfg
```

```
# Manual rescue from host:
qemu-nbd -c /dev/nbd0 linux-vm.qcow2
mount /dev/nbd0p3 /mnt/rescue
mount /dev/nbd0p1 /mnt/rescue/boot
mount --bind /dev /mnt/rescue/dev
mount --bind /proc /mnt/rescue/proc
mount --bind /sys /mnt/rescue/sys
chroot /mnt/rescue grub2-mkconfig -o /boot/grub2/grub.cfg
umount -R /mnt/rescue && qemu-nbd -d /dev/nbd0
```

UEFI & Firmware Issues

Mismatched firmware type is the #3 most common boot failure.

Firmware Detection & Configuration

Indicator	BIOS (Legacy)	UEFI	How h2kvmctl Detects	If Wrong
Partition table	MBR (msdos)	GPT	<code>parted -s /dev/nbd0 print grep "Partition Table"</code>	Wrong firmware = no boot
Boot partition	/boot (ext4/xfs)	/boot/efi (vfat, ESP)	Checks for EFI System Partition	BIOS can't read ESP, UEFI needs it
Windows boot	bootmgr (MBR)	bootmgfw.efi	Checks for EFI\Microsoft\Boot	BSOD or "no bootable device"
VMware source	<code>firmware="bios"</code>	<code>firmware="efi"</code>	Reads OVF descriptor	Auto-detected from OVF/OVA

Fixing Firmware Mismatches

Libvirt UEFI Configuration

```
<!-- UEFI boot (for GPT/EFI VMs) -->
<os>
  <type machine='pc-q35-8.2'>hvm</type>
  <loader readonly='yes' type='pflash'>
    /usr/share/OVMF/OVMF_CODE.fd
  </loader>
  <nvram>
    /var/lib/libvirt/qemu/nvram/vm_VARS.fd
  </nvram>
</os>

<!-- BIOS boot (for MBR VMs) -->
<os>
  <type machine='pc-i440fx-8.2'>hvm</type>
  <boot dev='hd' />
</os>
```

Secure Boot

- VMware EFI VMs may have Secure Boot enabled
- KVM supports Secure Boot with OVMF + enrolled keys
- h2kvmctl detects Secure Boot from OVF metadata
- Uses `OVMF_CODE.secboot.fd` firmware for Secure Boot
- Windows 11 requires Secure Boot + vTPM
- vTPM: `swtpm` package provides virtual TPM 2.0
- If Secure Boot causes issues: disable in OVMF setup (F2 at boot)
- `<os firmware='efi'><firmware><feature enabled='yes' name='secure-boot' /></firmware></os>`

h2kvmctl Doctor Output

Pre-flight system readiness check — run before any migration.

Doctor Check Areas

```
$ h2kvmctl doctor

System Readiness Check
=====

[PASS] KVM kernel module loaded (kvm_intel)
[PASS] /dev/kvm exists and accessible
[PASS] QEMU version 8.2.0 (minimum 6.0)
[PASS] libvirtd running (version 10.0.0)
[PASS] qemu-img available (version 8.2.0)
[PASS] qemu-nbd available
[PASS] VirtIO drivers: /usr/share/virtio-win/virtio-win.iso (0.1.240)
[PASS] OVMF firmware: /usr/share/OVMF/OVMF_CODE.fd
[PASS] swtpm available (for Windows 11 vTPM)
[PASS] Bridge br0 exists (state: UP, IP: 10.0.1.1/24)
[PASS] Free disk space: 850 GB on /var/lib/libvirt/images/
[WARN] NBD module not loaded — run: modprobe nbd max_part=8
[PASS] Python 3.11 with required packages
[PASS] SELinux: enforcing (contexts correct)

Result: 13/14 checks passed, 1 warning
Ready for migration (fix warnings for optimal operation)
```

Doctor Check Details

#	Check	What It Verifies	Common Failure	Fix
1	KVM kernel module	kvm + kvm_intel or kvm_amd loaded	VM host doesn't support VT-x/AMD-V	Enable virtualization in BIOS
2	/dev/kvm access	Device exists and user has access	Missing kvm group membership	<code>usermod -aG kvm \$USER</code>
3	QEMU version	qemu-system-x86_64 version >= 6.0	Old RHEL 7 QEMU (2.12)	Upgrade to RHEL 9 / Rocky 9
4	libvirtd running	Service active and socket available	Not installed or not started	<code>dnf install libvirt && systemctl enable --now libvirtd</code>

5	qemu-img/qemu-nbd	Conversion tools available	qemu-img-extra not installed	<code>dnf install qemu-img</code>
6	VirtIO drivers	virtio-win.iso exists	Not installed (needed for Windows)	<code>dnf install virtio-win</code>
7	OVMF firmware	UEFI firmware files exist	Missing edk2-ovmf package	<code>dnf install edk2-ovmf</code>
8	Disk space	>= 2x largest VM size free	Not enough space for conversion	Free space or use different output dir
9	Network bridge	At least one bridge exists and is UP	No bridge configured	Create bridge with nmcli (see network guide)

Conversion **Errors**

Diagnosing qemu-img failures, corrupt source images, and space issues.

Error: qemu-img convert failed

```
# Check source image integrity
qemu-img check source.vmdk
# Result: image corrupt / leaked clusters

# Fix: try to repair
qemu-img check --repair=all source.vmdk

# If still failing: extract with verbose
qemu-img convert -f vmdk -O qcow2 \
  -p source.vmdk output.qcow2 2>&1 | tee convert.log

# Common causes:
# 1. Incomplete OVA extract (truncated)
# 2. VMDK descriptor references missing extents
# 3. Snapshot chain not fully resolved
# 4. Source still in use (lock file)
```

Error: Insufficient disk space

```
# Check space before conversion
qemu-img info --output=json source.vmdk | \
  jq '["virtual-size"]'
# 107374182400 (100 GB virtual)

# Actual size on disk (thin provisioned)
qemu-img info source.vmdk | grep "disk size"
# disk size: 42 GB

# With compression, output typically 30-50% of actual
# Need: ~42 GB free (or ~25 GB with --compress)

# Use different output directory if needed
h2kvmctl convert --source vm.vmdk \
  --output /data/large-disk/images/

# Or convert to compressed QCOW2
h2kvmctl convert --source vm.vmdk \
  --compress # Saves 40-60% space
```

Error: OVA extraction failed

- **Symptom:** tar extraction fails or produces 0-byte files
- **Cause:** OVA was downloaded incompletely or corrupted
- **Verify:** `tar tf export.ova` should list `.ovf` + `.vmdk`
- **Check size:** Compare file size to expected (from vSphere)
- **SHA256:** Verify checksum if available from export
- **Re-export:** If corrupt, export again from VMware
- **govc:** `govc export.ovf -vm /DC/vm/name` (more reliable)

Error: VMDK descriptor invalid

- **Symptom:** "Could not open: unsupported VMDK format"
- **Cause 1:** VMDK is sesparse (vSphere snapshot format)
- **Fix 1:** Consolidate snapshots in VMware before export
- **Cause 2:** Split VMDK (2 GB extents)
- **Fix 2:** Use the descriptor file (`-flat.vmdk` referenced)
- **Cause 3:** VMDK from ESXi datastore (monolithic flat)
- **Fix 3:** Copy both descriptor + flat file together
- **Verify:** `head -5 vm.vmdk` should show descriptor

Network & Performance Issues

Post-migration network and performance troubleshooting.

Network Issues After Migration

No Network Connectivity

```
# Check 1: Is the NIC visible in guest?
ip link show
# If only 'lo' visible: VirtIO driver missing

# Check 2: Wrong interface name
# VMware: ens192 or eth0
# KVM: eth0, ens3, or enp1s0
# Fix: rename or update network config

# Check 3: Bridge not configured
virsh domiflist vm-name
# If bridge is wrong or missing:
virsh attach-interface vm-name bridge br0 \
  --model virtio

# Check 4: IP address not assigned
# DHCP: should auto-assign (same MAC)
# Static: update config in guest
# NetworkManager: nmcli con up "Wired 1"
```

Slow Network Performance

```
# Check: using virtio NIC (not e1000)
virsh dumpxml vm-name | grep "model type"
# Should be: <model type='virtio' />
# Not: <model type='e1000' /> (10x slower)

# Check: vhost-net enabled
lsmod | grep vhost_net
# If not: modprobe vhost_net

# Check: MTU mismatch
ip link show br0 # host bridge MTU
# Inside guest:
ip link show eth0 # guest MTU
# Must match! Fix: ip link set eth0 mtu 1500

# Benchmark
iperf3 -c 10.0.1.1 -t 30
# Expected: >8 Gbps on 10GbE with virtio
```

Performance Troubleshooting

Issue	Symptom	Diagnosis	Fix
CPU contention	High steal time in guest	top → %st column > 5%	Pin vCPUs: virsh vcpupin vm 0 2; avoid overcommit
Memory balloon	Guest sees less RAM than configured	virsh dommemstat vm	Disable balloon: <memballoon model='none' />
Disk I/O slow	High iowait, slow writes	iostat -x 1 in guest	Use cache=none + io=native; check disk bus=virtio
NUMA mismatch	40-70ns memory latency	numastat -p \$(pgrep qemu)	Pin vCPUs + memory to same NUMA node

No hugepages

TLB misses, higher latency

`grep HugePages /proc/meminfo`

Enable hugepages in domain XML

Log Analysis & Escalation

Where to find logs and how to interpret h2kvmctl output.

Log Locations

Component	Log File	What It Contains
h2kvmctl	<code>/var/log/hyper2kvm/migration.log</code>	Conversion steps, errors, timing, file operations
h2kvmctl (verbose)	<code>h2kvmctl convert --verbose 2>&1 tee debug.log</code>	Full debug output including qemu-img commands
libvirtd	<code>/var/log/libvirt/qemu/vm-name.log</code>	QEMU startup, device errors, shutdown
QEMU monitor	<code>virsh qemu-monitor-command vm --hmp info status</code>	Real-time VM state, device info
Guest console	<code>virsh console vm-name</code>	Boot messages, kernel output, login prompt
Guest serial	Add <code>console=ttyS0</code> to kernel cmdline	Serial console for headless debugging
systemd journal	<code>journalctl -u libvirtd -f</code>	libvirt service messages
Audit	<code>ausearch -m VIRT_CONTROL -ts recent</code>	VM start/stop/migrate audit events

Troubleshooting Decision Tree

VM Won't Start

- 1. Check `virsh start vm 2>&1` error message
- 2. Common: "Cannot access storage file" → check permissions
- 3. Common: "unsupported configuration" → check XML syntax
- 4. UEFI: verify OVMF firmware path exists
- 5. SELinux: `sealert -a /var/log/audit/audit.log`
- 6. AppArmor (Ubuntu): `aa-complain /etc/apparmor.d/*libvirt*`
- 7. Resource: `virsh freecell` — enough memory?

VM Starts But Won't Boot

- 1. Open console: `virt-viewer vm-name`
- 2. Black screen → firmware mismatch (BIOS vs UEFI)
- 3. GRUB prompt → disk layout changed, rebuild GRUB
- 4. Kernel panic → `initramfs` missing `virtio` (`--fix-initramfs`)
- 5. BSOD → Windows missing VirtIO drivers
- 6. Drops to dracut shell → `fstab` issue (`--fix-fstab`)
- 7. Still stuck? Boot rescue ISO: `--cdrom /path/rescue.iso`

95% of Issues Solved by: `--fix-initramfs --fix-fstab --virtio-drivers`

Run `h2kvmctl doctor` before migration. Use `--fix-initramfs --fix-fstab--fix-network` for Linux VMs and `--virtio-drivers` for Windows VMs. These four flags prevent the top 5 failure modes. For everything else, check the decision tree above and the log files. Most issues are diagnosed within 5 minutes.